

UNIT 4: FILE INDEXING AND EXTERNAL SEARCHING

Concepts, File Indexing, Performance

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS



Course Outline

- 1 General Concepts
- 2 File Indexing
- 3 Performance & Algorithms
- 4 Algorithms



Outline

- 1 General Concepts
- 2 File Indexing
- 3 Performance & Algorithms
- 4 Algorithms



External Storage Foundations I

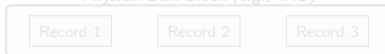
File Organization

The physical and logical arrangement of data records within a secondary non-volatile storage file, determining how blocks are allocated and mapped.

Blocks and Records

- **Record:** The logical unit of user data collection.
- **Block (Page):** The minimum physical data unit transferred during a single disk input/output (I/O) hardware operation.

Physical Disk Block (e.g., 4KB)



External Storage Foundations I

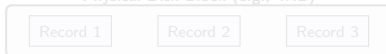
File Organization

The physical and logical arrangement of data records within a secondary non-volatile storage file, determining how blocks are allocated and mapped.

Blocks and Records

- **Record:** The logical unit of user data collection.
- **Block (Page):** The minimum physical data unit transferred during a single disk input/output (I/O) hardware operation.

Physical Disk Block (e.g., 4KB)



External Storage Foundations I

File Organization

The physical and logical arrangement of data records within a secondary non-volatile storage file, determining how blocks are allocated and mapped.

Blocks and Records

- **Record:** The logical unit of user data collection.
- **Block (Page):** The minimum physical data unit transferred during a single disk input/output (I/O) hardware operation.

Physical Disk Block (e.g., 4KB)



External Storage Foundations II

External Search Constraint

Main memory CPU operations take nanoseconds, while disk seeking takes milliseconds. Optimization means minimizing **Block Reads**, not comparisons.



Outline

- 1 General Concepts
- 2 File Indexing**
- 3 Performance & Algorithms
- 4 Algorithms



File Indexing Architectures

Indexing

An auxiliary data structure containing ordered (**Key, Pointer**) tuples used to find the exact disk block containing target records without a full sequential file scan.

Primary Indexes

Built directly on a file's **ordered primary key field**. Typically sparse, containing one entry per data block.

Secondary Indexes

Built on **unordered non-primary fields**. Dense structure containing an individual entry for every distinct record pointer.

Multilevel Indexes

When an index file grows too large for RAM, a secondary index is built on top of the primary index, creating a search tree structure (e.g., B+ Trees).



File Indexing Architectures

Indexing

An auxiliary data structure containing ordered (**Key**, **Pointer**) tuples used to find the exact disk block containing target records without a full sequential file scan.

Primary Indexes

Built directly on a file's **ordered primary key field**. Typically sparse, containing one entry per data block.

Secondary Indexes

Built on **unordered non-primary fields**. Dense structure containing an individual entry for every distinct record pointer.

Multilevel Indexes

When an index file grows too large for RAM, a secondary index is built on top of the primary index, creating a search tree structure (e.g., B+ Trees).



File Indexing Architectures

Indexing

An auxiliary data structure containing ordered (**Key, Pointer**) tuples used to find the exact disk block containing target records without a full sequential file scan.

Primary Indexes

Built directly on a file's **ordered primary key field**. Typically sparse, containing one entry per data block.

Secondary Indexes

Built on **unordered non-primary fields**. Dense structure containing an individual entry for every distinct record pointer.

Multilevel Indexes

When an index file grows too large for RAM, a secondary index is built on top of the primary index, creating a search tree structure (e.g., B+ Trees).



File Indexing Architectures

Indexing

An auxiliary data structure containing ordered (**Key**, **Pointer**) tuples used to find the exact disk block containing target records without a full sequential file scan.

Primary Indexes

Built directly on a file's **ordered primary key field**. Typically sparse, containing one entry per data block.

Secondary Indexes

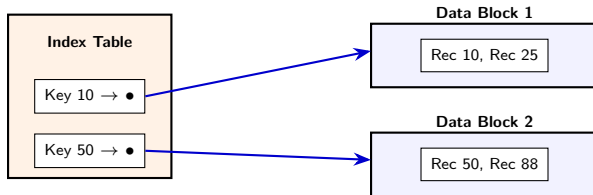
Built on **unordered non-primary fields**. Dense structure containing an individual entry for every distinct record pointer.

Multilevel Indexes

When an index file grows too large for RAM, a secondary index is built on top of the primary index, creating a search tree structure (e.g., B+ Trees).



Primary Index Structure Lookup



Structural Property

Binary searching the sparse Index Table finds the correct block address instantly, requiring only a single data block I/O fetch operation.



Outline

- 1 General Concepts
- 2 File Indexing
- 3 Performance & Algorithms**
- 4 Algorithms



Performance Evaluation & Operational Costs

Disk Access Cost Formulations

Let b be the total data block count, and b_i be the index block count.

- **Unindexed Search (Sequential):** Requires $O(b)$ disk reads.
- **Primary Indexed Search:** Requires $O(\log_2 b_i) + 1$ disk reads.

External File Search Metrics

Index Mode	Search I/O Cost	Insertion Cost	Storage Space
No Index	$b/2$ Avg Block Reads	$O(1)$ Append	Zero Overhead
Primary (Sparse)	$\log_2 b_i + 1$ Blocks	High (Shifting)	Very Low
Secondary (Dense)	$\log_2 b_i + 1$ Blocks	High Overhead	High
Multilevel Tree	$\log_{\text{fanout}} b$	Balanced Split	Adaptive



Performance Evaluation & Operational Costs

Disk Access Cost Formulations

Let b be the total data block count, and b_i be the index block count.

- **Unindexed Search (Sequential):** Requires $O(b)$ disk reads.
- **Primary Indexed Search:** Requires $O(\log_2 b_i) + 1$ disk reads.

External File Search Metrics

Index Mode	Search I/O Cost	Insertion Cost	Storage Space
No Index	$b/2$ Avg Block Reads	$O(1)$ Append	Zero Overhead
Primary (Sparse)	$\log_2 b_i + 1$ Blocks	High (Shifting)	Very Low
Secondary (Dense)	$\log_2 b_i + 1$ Blocks	High Overhead	High
Multilevel Tree	$\log_{\text{fanout}} b$	Balanced Split	Adaptive



Performance Evaluation & Operational Costs

Core Algorithmic Trade-off

Indexes dramatically speed up queries ($\downarrow$ Disk Reads) but increase the performance cost of updates, insertions, and deletions ($\uparrow$ Structure Maintenance Cost).



Outline

- 1 General Concepts
- 2 File Indexing
- 3 Performance & Algorithms
- 4 Algorithms



Index Construction & Search Operations I

1. Index Construction

How indexing structures are initially generated:

- **Step 1:** Linearly scan the physical data blocks of the main storage file.
- **Step 2:** Extract the key attribute (for primary indexes, pick the first key entry of each block).
- **Step 3:** Write the sorted (**Key**, **Block_Pointer**) pair directly into the dedicated index file.



Index Construction & Search Operations II

2. Search Operations

The exact mechanical lookup routine:

- 1 Perform a binary search on the loaded **Index Table** elements.
- 2 Identify the highest key value less than or equal to the target search key.
- 3 Extract the mapped disk block memory pointer address.
- 4 Issue a single **Disk Read I/O command** to pull that specific block context into RAM for final internal extraction.



Update Mechanisms & Structure Maintenance

The Challenge of Data Modifications

While indexing speeds up searches, any changes to raw data items (insertions, updates, or deletions) trigger mandatory synchronization steps.

Update Mechanisms Categories:

- **Record Insertion:** Requires finding the correct physical block. If the target block is full, it causes a costly physical **Block Split** or requires writing to an un-indexed overflow bucket area.
- **Record Deletion:** The item is purged from the data block. If a sparse primary key index entry is removed, the index file itself must be rewritten.
- **Index Reindexing:** Periodic background routines that reorganize pointers to eliminate dead leaf gaps and fix disk fragmentation.



Update Mechanisms & Structure Maintenance

The Challenge of Data Modifications

While indexing speeds up searches, any changes to raw data items (insertions, updates, or deletions) trigger mandatory synchronization steps.

Update Mechanisms Categories:

- **Record Insertion:** Requires finding the correct physical block. If the target block is full, it causes a costly physical **Block Split** or requires writing to an un-indexed overflow bucket area.
- **Record Deletion:** The item is purged from the data block. If a sparse primary key index entry is removed, the index file itself must be rewritten.
- **Index Reindexing:** Periodic background routines that reorganize pointers to eliminate dead leaf gaps and fix disk fragmentation.



Outline

- 1 General Concepts
- 2 File Indexing
- 3 Performance & Algorithms
- 4 Algorithms



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

